

Oefentoets SDK en SAAI

Disclaimer: deze oefentoets is een indicatie voor de soort vragen die op de toets naar voren komen, geen representatie van de exacte inhoud (bv welke design patterns) of hoe vaak een bepaald onderwerp wordt behandeld.

Software Design en Kwaliteit (max 50 punten)

Onderwerpen: de SDK workshops van de eerste twee weken (gelabeld met LU1 op Brightspace)

Vragen:

1. In een computerspel komen vele soorten monsters voor. Er zijn vele subklassen van Monster, zoals Spider, Duck, Tiger etc.
Elk van deze monsters komt voor in verschillende 'typen'. Elk type behoort tot een reeks (Engels: line). Zo heb je de frosty-line, fiery-line, gold-line etc. Er bestaat dus een FrostySpider, FrostyDuck, FierySpider, FieryDuck, etc.

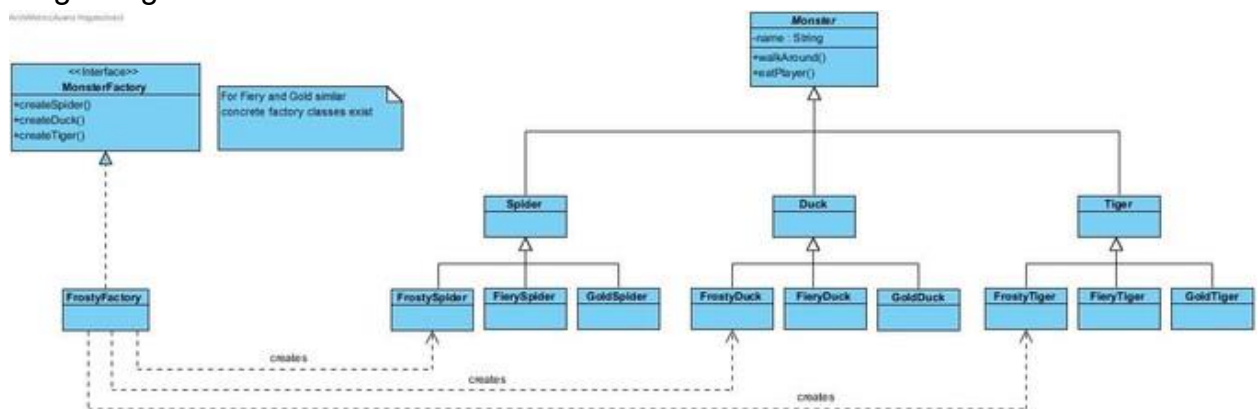
We maken gebruik van het Abstract Factory Pattern om monsters van een bepaald type te creëren.

Hieronder is een gedeelte gegeven van het model, zoals we dat gaan toepassen. Het model is nog niet compleet.

Vraag:

We willen een nieuw Monster toevoegen: Gorilla. in een nieuwe reeks: silver (de silverline) Beschrijf welke klassen of interfaces we moeten toevoegen of wijzigen in het ontwerp.

Geef duidelijk aan waar een klasse, attribuut of methode moet worden toegevoegd.



2. Welke van de onderstaande beweringen zijn waar?

A) Loose coupling is een concept dat uitbreiding van software eenvoudiger maakt

B) Loose coupling is noodzakelijk om het open/closed principe te kunnen toepassen

- ☐ Bewering A en B zijn beiden waar.
- ☒ Bewering A is waar en bewering B is **niet** waar.
- ☐ Bewering A is **niet** waar en bewering B is waar.
- ☐ Beide beweringen zijn **niet** waar.

3. Gegeven is (een deel van) het ontwerp van de bibliotheek-casus.

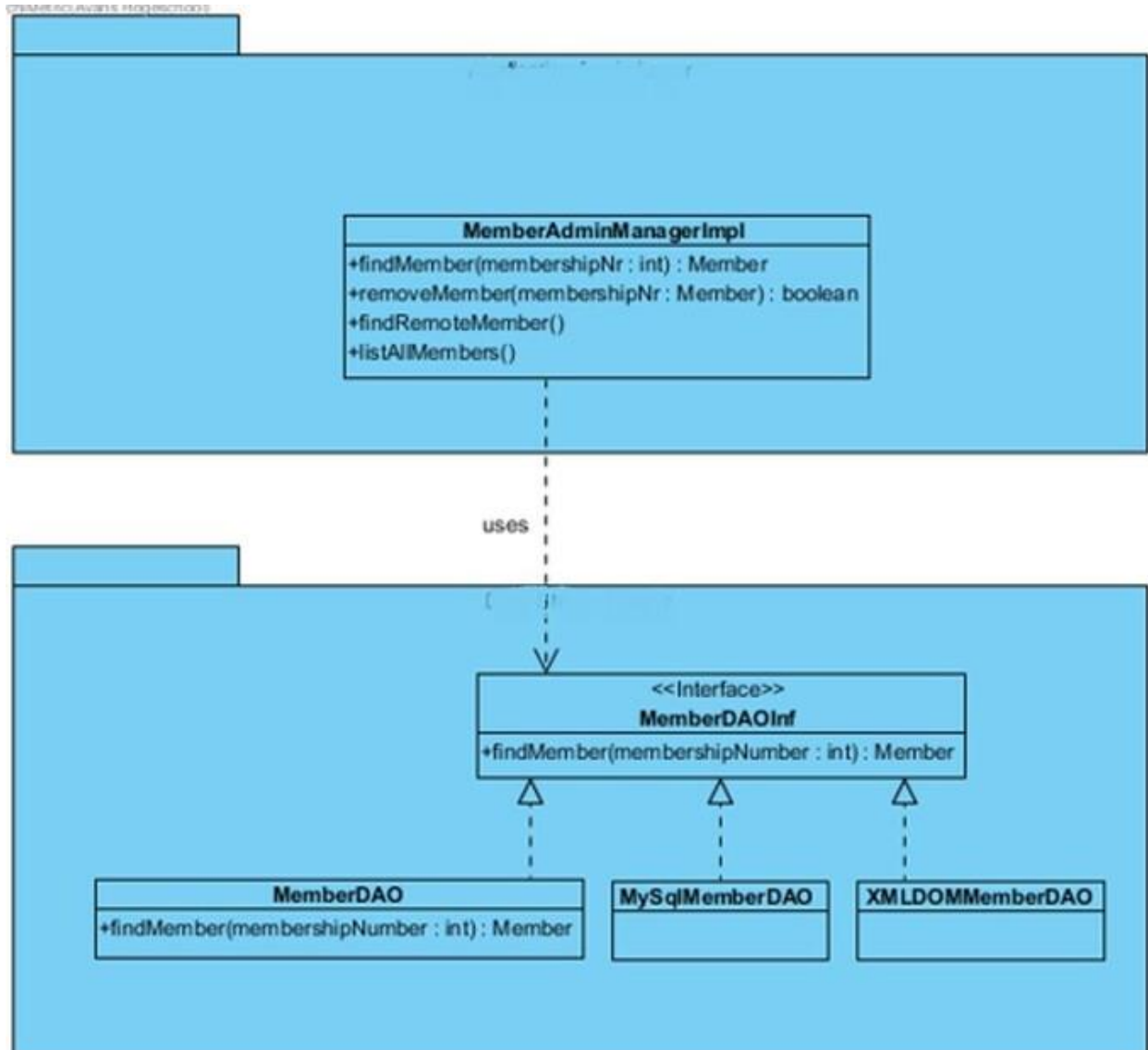
Geef de (pseudo) code voor de main klasse zodat loose coupling voor de MemberAdminManagerImpl klasse tot zijn recht komt. Laat zien wat je hiervoor moet doen in:

* de klasse MemberAdminManagerImpl

* de Main klasse om de MemberAdminManagerImpl op de juiste wijze te instantieëren.

(Let op, je wordt niet beoordeeld op foutloze syntax, laat duidelijk zien hoe loose coupling tot stand

komt!)



4. Leg in eigen woorden uit wat het Single Responsibility Principle betekent en waarom je dit toepast.

Antwoorden:

1. Een nieuwe superklasse Gorilla

Subklassen FrostyGorrilla, FieryGorrilla..etc

Toevoegen van nieuwe klassen silverSpider, silverDuck en silverTiger, silverGorilla, .
Deze erven van hun generieke superklasse: Spider|Duck|Tiger|Gorilla

Toevoegen methode createGorilla aan interface MonsterFactory

Deze methode in de bestaande implementatieklassen (FrostyFactory etc.)
implementeren

En we moeten een Factory toevoegen om Monsters van dit type te genereren:
silverFactory.

Deze erft van de MonsterFactory-interface en implementeert de methoden:
createSpider(), createDuck(), createTiger() en createGorilla voor de silver-uitvoeringen
van de monsters.

2. Bewering A is waar en Bewering B is **niet** waar, omdat:

A) Loose coupling zorgt ervoor dat een klasse niet afhankelijk is van een andere concrete klasse. Dit kun je enkel bereiken door het toepassen van een abstractie (interface of abstracte klasse) tussen de twee klassen die je wil koppelen (dependency inversion). Dit principe wordt ook wel eens 'program to an interface' genoemd. Door dit principe kun je gemakkelijker in de toekomst uitbreidingen maken volgens het open/closed principe.

B) Loose coupling is niet noodzakelijk om het open/closed principe te kunnen toepassen. Als je een interface toepast op verschillende implementaties van concrete klassen voldoe je aan het open/closed principe. Ook als je in de klasse die dit gebruik de concrete implementatie instantieert. Je hebt dat wel het open/closed principe toegepast, maar dus geen loose coupling.

3. In Main klasse:

```
public static main() {  
    MemberDaoInf md = new MySQLMemberDAO();  
    MemberAdminManagerImp manager = new MemberAdminManagerImp(md);  
}
```

In MemberAdminManagerImp:

1. Attribuut van type MemberDAOInf toevoegen
2. Constructor:

```
public MemberAdminManagerImp(MemberDAOInf memberDAO) {  
    this.memberDAO = memberDAO;  
}
```

4. Het Single Responsibility Principle (SRP) schrijft voor dat elke 'unit' slechts één verantwoordelijkheid mag hebben.

Dit betekent:

- Bij methoden: een methode doet één enkel ding, dus een methode SortGrocerieslistAndPrint() doet te veel en kan beter gesplitst worden in twee methoden SortGrocerieslist() en PrintGrocerieslist().
- Bij Klassen: een klasse is verantwoordelijk voor één enkel ding, Bij een facturatieprogramma gaat een klasse OrderLine waaruit een Order is opgebouwd niets doen met het factuuradres of het optellen van de prijs van de totale order.
- Bij Packages en/of applicatie-architectuurlagen: een GUI package of architectuurlaag gaat geen gegevens ophalen uit een database.

Je past het SRP toe zodat je logica voor een bepaalde functionaliteit bij elkaar houdt. Je voorkomt hiermee 'spaghetti-code' en klassen of methoden die gigantisch groot worden. Je zorgt ervoor dat de onderhoudbaarheid van je applicatie verbeterd wordt.

Software Architectuur en Applicatie Integratie (max 50 punten)

In de toets komt een casus die lijkt op de groepsopdracht: een legacy applicatie, met een nieuwe module die gekoppeld is aan een extern system.

Gevraagd wordt om twee ADR's te schrijven (Nygard formaat, indeling is gegeven) die toekomstige kwaliteit van het bestaande software systeem verhoogd.

De beoordeling vindt plaats op de toepassing van de onderwerpen uit de eerste twee workshops (systeemanalyse / messages & events).